

SLOT5: Quy tắc đặt tên Collection

Trong các ứng dụng sử dụng **Node.js** và **Express.js** kết hợp với MongoDB, việc đặt tên **collection** cần được thực hiện sao cho tương thích với cấu trúc **schema** (thường được định nghĩa bởi thư viện như **Mongoose**) và tuân theo các quy ước lập trình hiện đại. Dưới đây là hướng dẫn chi tiết về cách đặt tên collection trong MongoDB để phù hợp với schema trong Node.js/Express.js:

1. Nguyên tắc đặt tên collection tương ứng với schema

- **Sử dụng tên số nhiều:**

- Trong MongoDB, collection thường đại diện cho một nhóm document, nên tên collection nên ở dạng số nhiều, tương ứng với tên model trong schema.
- Ví dụ: Nếu schema định nghĩa một User, collection nên là users.
- Trong Mongoose, khi bạn định nghĩa một schema như sau:

javascript

```
const userSchema = new mongoose.Schema({
  name: String,
  age: Number
});
```

```
const User = mongoose.model('User', userSchema);
```

Mongoose sẽ tự động ánh xạ model User đến collection users (chuyển thành số nhiều và viết thường).

- **Tên collection khớp với tên model:**

- Tên collection nên phản ánh tên model trong schema để dễ nhận diện. Ví dụ:
 - Schema Product → Collection products
 - Schema Order → Collection orders
 - Schema Category → Collection categories

- **Sử dụng camelCase hoặc snake_case:**

- **camelCase:** Phổ biến trong JavaScript/Node.js, ví dụ: userProfiles, orderDetails.
- **snake_case:** Thường được dùng trong cơ sở dữ liệu, ví dụ: user_profiles, order_details.
- Chọn một kiểu đặt tên và sử dụng nhất quán. Trong Mongoose, bạn có thể chỉ định tên collection cụ thể nếu không muốn dùng mặc định:

javascript

```
const userSchema = new mongoose.Schema({ ... }, { collection:
  'user_profiles' });
const User = mongoose.model('User', userSchema);
```

- **Tránh ký tự đặc biệt:**

- Chỉ nên dùng chữ cái, số, dấu gạch dưới (_) hoặc gạch ngang (-). Tránh các ký tự như @, #, khoảng trắng, vì chúng có thể gây lỗi khi tích hợp với Node.js/Express.js.

2. Tích hợp với Mongoose trong Node.js/Express.js

Mongoose là thư viện phổ biến để làm việc với MongoDB trong Node.js. Khi đặt tên collection, bạn cần lưu ý cách Mongoose ánh xạ schema với collection:

- **Tự động chuyển thành số nhiều:**

- Mongoose tự động chuyển tên model thành dạng số nhiều và viết thường để tạo tên collection. Ví dụ:
 - Model User → Collection users
 - Model BlogPost → Collection blogposts
- Nếu muốn chỉ định tên collection khác, bạn có thể thêm tùy chọn collection trong schema:

javascript

```
const productSchema = new mongoose.Schema({
  name: String,
  price: Number
}, { collection: 'product_items' });
const Product = mongoose.model('Product', productSchema);
```

Ở đây, collection sẽ là product_items thay vì products.

- **Kiểm soát tên collection:**

- Nếu bạn muốn tên collection khớp chính xác với tên model (không chuyển thành số nhiều), hãy khai báo rõ ràng trong schema:

javascript

```
const userSchema = new mongoose.Schema({ ... }, { collection:
  'User' });
```

- **Kiểm tra collection trong MongoDB Shell:**

- Sau khi tạo schema và lưu dữ liệu, bạn có thể kiểm tra collection trong mongosh:

javascript

```
show collections
db.users.find()
```

3. Quy ước đặt tên trong dự án thực tế

- **Phân chia theo chức năng:**

- Nếu dự án có nhiều loại dữ liệu, đặt tên collection theo chức năng hoặc module. Ví dụ:
 - Module người dùng: users, user_sessions, user_preferences
 - Module sản phẩm: products, product_reviews, product_categories
 - Module đơn hàng: orders, order_items

- **Tên ngắn gọn, rõ nghĩa:**

- Tránh tên quá dài hoặc mơ hồ. Ví dụ:
 - Tốt: users, orders, products
 - Không tốt: all_user_data, temporary_order_storage
- **Hỗ trợ đa ngôn ngữ hoặc đặc thù dự án:**
 - Nếu dự án cần tên collection phù hợp với ngôn ngữ cụ thể (ví dụ tiếng Việt), bạn có thể dùng tên tiếng Anh để đảm bảo tính phổ quát, hoặc dùng tên tiếng Việt không dấu:
 - Ví dụ: nguoi_dung thay vì người dùng.

4. Ví dụ thực tế với Node.js/Express.js

Dưới đây là ví dụ về cách định nghĩa schema và ánh xạ collection trong một ứng dụng Express.js sử dụng Mongoose:

javascript

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();

// Kết nối MongoDB
mongoose.connect('mongodb://localhost:27017/mydb', { useNewUrlParser:
true, useUnifiedTopology: true });

// Định nghĩa schema và model
const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
}, { collection: 'users' }); // Chỉ định tên collection

const User = mongoose.model('User', userSchema);

// API để thêm user
app.post('/users', async (req, res) => {
  const user = new User({
    name: req.body.name,
    email: req.body.email,
    age: req.body.age
  });
  await user.save();
  res.send('User added to users collection');
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

- Trong ví dụ trên, collection users được sử dụng để lưu trữ dữ liệu người dùng.
- Nếu không chỉ định { collection: 'users' }, Mongoose sẽ tự động tạo collection users từ model User.

5. Lưu ý khi làm việc với collection

- **Kiểm tra tồn tại collection:**
 - Trong MongoDB Shell: db.getCollectionNames()
 - Trong Mongoose, bạn có thể kiểm tra collection bằng cách truy vấn:

javascript

```
User.find().then(data => console.log(data));
```

- **Xử lý tên không hợp lệ:**
 - Nếu tên collection không hợp lệ (ví dụ, chứa khoảng trắng), Mongoose sẽ báo lỗi. Hãy đảm bảo tuân theo quy tắc đặt tên của MongoDB (xem câu trả lời trước).
- **Hiệu suất:**
 - Tên collection ngắn gọn giúp giảm chi phí lưu trữ và truy vấn, đặc biệt trong các ứng dụng lớn.

6. Tóm tắt quy ước đặt tên

- **Tên collection:** Số nhiều, viết thường, sử dụng camelCase hoặc snake_case.
- **Tương ứng với schema:** Khớp với tên model trong Mongoose (số nhiều tự động).
- **Nhất quán:** Sử dụng một quy ước duy nhất trong toàn bộ dự án.
- **Rõ ràng:** Tên nên phản ánh nội dung của collection (ví dụ: users, products).
- **Tùy chỉnh nếu cần:** Sử dụng tùy chọn collection trong schema Mongoose để kiểm soát tên collection.

CÁC CÁCH XỬ LÝ TRƯỜNG HỢP ĐẶT TÊN COLLECTION KHÔNG THEO QUY TẮC

Trong các ứng dụng sử dụng **Node.js** và **Express.js** kết hợp với MongoDB, việc đặt tên **collection** cần được thực hiện sao cho tương thích với cấu trúc **schema** (thường được định nghĩa bởi thư viện như **Mongoose**) và tuân theo các quy ước lập trình hiện đại. Dưới đây là hướng dẫn chi tiết về cách đặt tên collection trong MongoDB để phù hợp với schema trong Node.js/Express.js:

1. Nguyên tắc đặt tên collection tương ứng với schema

- **Sử dụng tên số nhiều:**

- Trong MongoDB, collection thường đại diện cho một nhóm document, nên tên collection nên ở dạng số nhiều, tương ứng với tên model trong schema.
- Ví dụ: Nếu schema định nghĩa một User, collection nên là users.
- Trong Mongoose, khi bạn định nghĩa một schema như sau:
javascript

```
const userSchema = new mongoose.Schema({
  name: String,
  age: Number
});
```

```
const User = mongoose.model('User', userSchema);
```

Mongoose sẽ tự động ánh xạ model User đến collection users (chuyển thành số nhiều và viết thường).

- **Tên collection khớp với tên model:**

- Tên collection nên phản ánh tên model trong schema để dễ nhận diện. Ví dụ:
 - Schema Product → Collection products
 - Schema Order → Collection orders
 - Schema Category → Collection categories

- **Sử dụng camelCase hoặc snake_case:**

- **camelCase:** Phổ biến trong JavaScript/Node.js, ví dụ: userProfile, orderDetails.
- **snake_case:** Thường được dùng trong cơ sở dữ liệu, ví dụ: user_profiles, order_details.
- Chọn một kiểu đặt tên và sử dụng nhất quán. Trong Mongoose, bạn có thể chỉ định tên collection cụ thể nếu không muốn dùng mặc định:
javascript

```
const userSchema = new mongoose.Schema({ ... }, { collection:
  'user_profiles' });
const User = mongoose.model('User', userSchema);
```

- **Tránh ký tự đặc biệt:**

- Chỉ nên dùng chữ cái, số, dấu gạch dưới (_) hoặc gạch ngang (-). Tránh các ký tự như @, #, khoảng trắng, vì chúng có thể gây lỗi khi tích hợp với Node.js/Express.js.

2. Tích hợp với Mongoose trong Node.js/Express.js

Mongoose là thư viện phổ biến để làm việc với MongoDB trong Node.js. Khi đặt tên collection, bạn cần lưu ý cách Mongoose ánh xạ schema với collection:

- **Tự động chuyển thành số nhiều:**

- Mongoose tự động chuyển tên model thành dạng số nhiều và viết thường để tạo tên collection. Ví dụ:
 - Model User → Collection users
 - Model BlogPost → Collection blogposts
- Nếu muốn chỉ định tên collection khác, bạn có thể thêm tùy chọn collection trong schema:
javascript

```
const productSchema = new mongoose.Schema({
  name: String,
  price: Number
}, { collection: 'product_items' });
const Product = mongoose.model('Product', productSchema);
```

Ở đây, collection sẽ là product_items thay vì products.

- **Kiểm soát tên collection:**

- Nếu bạn muốn tên collection khớp chính xác với tên model (không chuyển thành số nhiều), hãy khai báo rõ ràng trong schema:

```
const userSchema = new mongoose.Schema({ ... }, { collection:
  'User' });
```

- **Kiểm tra collection trong MongoDB Shell:**

- Sau khi tạo schema và lưu dữ liệu, bạn có thể kiểm tra collection trong mongosh:

```
show collections
db.users.find()
```

3. Quy ước đặt tên trong dự án thực tế

- **Phân chia theo chức năng:**

- Nếu dự án có nhiều loại dữ liệu, đặt tên collection theo chức năng hoặc module. Ví dụ:
 - Module người dùng: users, user_sessions, user_preferences
 - Module sản phẩm: products, product_reviews, product_categories
 - Module đơn hàng: orders, order_items

- **Tên ngắn gọn, rõ nghĩa:**

- Tránh tên quá dài hoặc mơ hồ. Ví dụ:
 - Tốt: users, orders, products
 - Không tốt: all_user_data, temporary_order_storage

- **Hỗ trợ đa ngôn ngữ hoặc đặc thù dự án:**

- Nếu dự án cần tên collection phù hợp với ngôn ngữ cụ thể (ví dụ tiếng Việt), bạn có thể dùng tên tiếng Anh để đảm bảo tính phổ quát, hoặc dùng tên tiếng Việt không dấu:
 - Ví dụ: nguoi_dung thay vì người dùng.

4. Ví dụ thực tế với Node.js/Express.js

Dưới đây là ví dụ về cách định nghĩa schema và ánh xạ collection trong một ứng dụng Express.js sử dụng Mongoose:

javascript

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();

// Kết nối MongoDB
mongoose.connect('mongodb://localhost:27017/mydb', { useNewUrlParser: true,
  useUnifiedTopology: true });

// Định nghĩa schema và model
const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
}, { collection: 'users' }); // Chỉ định tên collection

const User = mongoose.model('User', userSchema);

// API để thêm user
app.post('/users', async (req, res) => {
```

```
const user = new User({
  name: req.body.name,
  email: req.body.email,
  age: req.body.age
});
await user.save();
res.send('User added to users collection');
});
```

```
app.listen(3000, () => console.log('Server running on port 3000'));
```

- Trong ví dụ trên, collection users được sử dụng để lưu trữ dữ liệu người dùng.
- Nếu không chỉ định { collection: 'users' }, Mongoose sẽ tự động tạo collection users từ model User.

5. Lưu ý khi làm việc với collection

- **Kiểm tra tồn tại collection:**
 - Trong MongoDB Shell: `db.getCollectionNames()`
 - Trong Mongoose, bạn có thể kiểm tra collection bằng cách truy vấn: javascript

```
User.find().then(data => console.log(data));
```
- **Xử lý tên không hợp lệ:**
 - Nếu tên collection không hợp lệ (ví dụ, chứa khoảng trắng), Mongoose sẽ báo lỗi. Hãy đảm bảo tuân theo quy tắc đặt tên của MongoDB (xem câu trả lời trước).
- **Hiệu suất:**
 - Tên collection ngắn gọn giúp giảm chi phí lưu trữ và truy vấn, đặc biệt trong các ứng dụng lớn.

6. Tóm tắt quy ước đặt tên

- **Tên collection:** Số nhiều, viết thường, sử dụng camelCase hoặc snake_case.
- **Tương ứng với schema:** Khớp với tên model trong Mongoose (số nhiều tự động).
- **Nhất quán:** Sử dụng một quy ước duy nhất trong toàn bộ dự án.
- **Rõ ràng:** Tên nên phản ánh nội dung của collection (ví dụ: users, products).
- **Tùy chỉnh nếu cần:** Sử dụng tùy chọn collection trong schema Mongoose để kiểm soát tên collection.

7. Tài liệu tham khảo

- Tài liệu Mongoose: <https://mongoosejs.com/docs/models.html>
- MongoDB Collection Naming: <https://www.mongodb.com/docs/manual/core/databases-and-collections/>